# A Comprehensive Investigation of Natural Language Processing Techniques and Tools to Generate Automated Test Cases

Imran Ahsan[1], Wasi Haider Butt[2], Mudassar Adeel Ahmed[3] and Muhammad Waseem Anwar[4]

Department of Computer Engineering, College of E&ME, National University of Sciences and Technology (NUST), H-12, Islamabad, Pakistan

imran.ahsan14@ce.ceme.edu.pk , wasi@ceme.nust.edu.pk , mudassar.adeel14@ce.ceme.edu.pk , waseemanwar@ceme.nust.edu.pk

## ABSTRACT

Natural Language Processing (NLP) techniques show promising results to organize and identify desired information from the bulky raw data. As a result, NLP techniques are continuously getting researcher's attention to automate various software development activities like test cases generation. However, selection of right NLP techniques and tools to generate automated test cases is always challenging. Therefore, in this paper, we investigate the application of NLP techniques to generate test cases from preliminary requirements document. A Systematic Literature Review (SLR) has been conducted to identify 16 research works published during 2005-2014. Consequently, 6 NLP techniques and 18 tools have been identified. Furthermore, 4 test case generation approaches and 9 NLP algorithms have also been presented. The identified NLP techniques and tools are highly beneficial for the researchers and practitioners of the domain.

## CCS Concepts

• **Computing methodologies~Natural language processing** • *Computing methodologies~Information extraction* • *Software and its engineering~Software verification and validation*

## Keywords

NLP; SLR; Text Mining; Test Cases; Test Case Generation; Nature Language Processing

## 1. INTRODUCTION

Getting desired information from preliminary data requires intensive manual operations that leads to huge data processing time. Furthermore, such manual data processing is more prone to classification errors especially in case of huge and complex data. To overcome such issues, Natural Language Processing (NLP) techniques show promising results, particularly, in the bio-medical domain [28]. As NLP techniques provide sophisticated and automated data processing features, it is now widely applied in software development to automatically generate requirements [29], design artifacts [30] and test cases [3] from preliminary data.


ICC '17, March 22 2017, Cambridge, United Kingdom

DOI: http://dx.doi.org/10.1145/3018896.3036375

Although NLP has been widely applied to automate software development activities, it is still hard to find right techniques and tools to generate automated test cases from initial requirements. The primary reason is the complexity of test case generation as the objective is to produce executable output rather to only classify / identify text and documents from the data. To the best of our knowledge, no real efforts have been made to summarize the existing research works in the context of text cases generation through NLP. Therefore, in this paper, we investigate NLP techniques and tools to generate automated test cases from initial requirements document. The major NLP steps involved in the test case generation are shown in **Figure 1**.

Initial requirements document, written in natural language / plain text, is normally used to generate test cases. Four major NLP steps (i.e. Tokenization [4], POS tagging [2], Chunking and Parsing [3]) are applied on initial requirements document to generate test cases as shown in Figure. It is important to note that NLP four steps can be applied collectively or individually depending upon the test case generation requirements. Finally, generated test cases are validated through the domain experts.

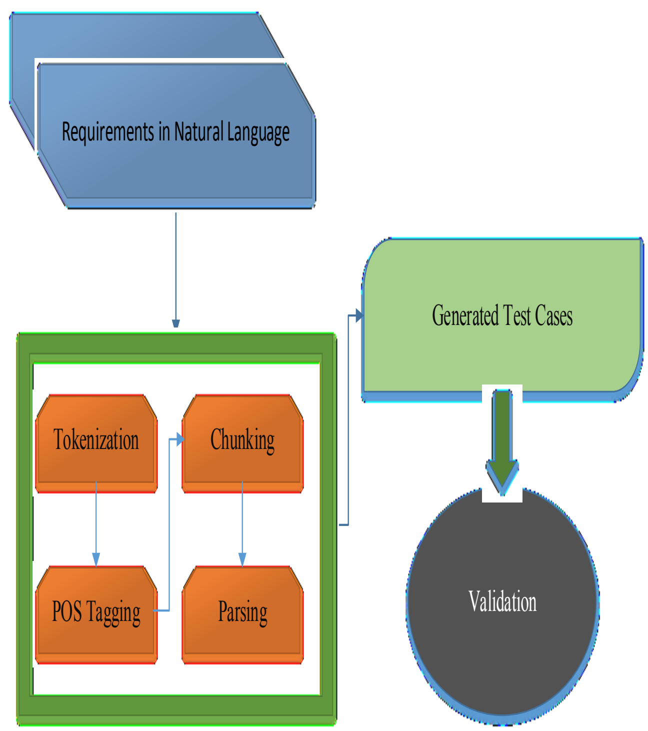


**Figure 1**. **Major NLP steps to generate test cases**

A Systematic Literature Review (SLR) has been used as a research methodology. Following research questions have been developed to carry out SLR:

**RQ1**: What research works have been reported from 2005 to 2016 where NLP techniques have been utilized to generate automated test cases from initial requirements?

**RQ2**: How major NLP steps like Tokenization, POS tagging, Chunking and Parsing have been applied for test cases generation?

**RQ3**: What are the leading NLP tools and algorithms, utilized or proposed by the researchers, in order to generate test cases from initial requirements?

The overview of research has been shown in **Figure 2**. We select three scientific databases (i.e. ACM [31], IEEE [32] and SPRINGER [33]) to carry out this research as per review protocol (Section **2.1**). On the basis of inclusion / exclusion criteria (Section **2.1.1**), we select 16 research works (Section **3.1**) pertaining to NLP in the context of automated test generation. We perform comprehensive investigation of selected research works and identified 18 NLP tools (Section **3.4**) as shown in **Figure 2**. The significant aspects of research have been discussed in Section **4** and paper is concluded in Section **7**.

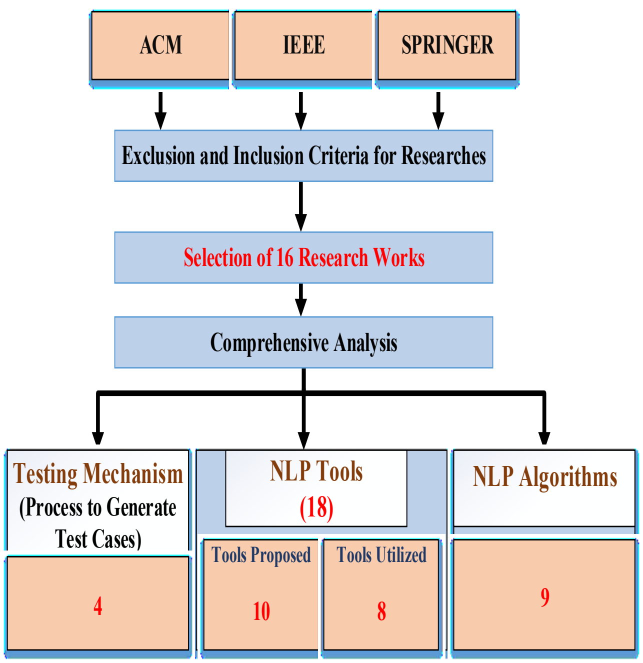


**Figure 2. Overview of research**

## 2. RESEARCH METHODOLOGY

This research has been carried out through Systematic Literature Review (SLR) [27]. It is an appropriate and proper process to document and address the relevant details precisely on the particular research area by investigating and reviewing all the existing research studies which are relevant to the research questions. So, there are five major stages of this research study: 1) Development of review protocol 2) Criteria of inclusion and exclusion 3) Search process 4) Quality Assessment 5) Extraction and synthesis of data.

### 2.1 Review Protocol Development

We develop a review protocol for the selection of research works. It defines background of the research, research questions relevant to the topic, inclusion and exclusion criteria, process of search, quality assessment, data extraction and synthesis. Background and research questions are already addressed in the introduction (Section 1). Other details are given in the subsequent sections.

#### *2.1.1 Criteria for Inclusion/Exclusion of Researches*

Specific criteria is defined for the inclusion and exclusion of research work. We defined 6 parameters as follows:

**Relevant to the subject:** Only include those researches that addressed the answers for our research questions.

**2005–2016:** All the included researches must be from 2005 to 2016. All other earlier researches should be excluded because only latest researches are included.

**Publisher:** Only those researches are included which are published in ACM, IEEE and Springer.

**Crucial-effects:** There should be some crucial effects regarding the NLP outputs (i.e. generated test cases) included in the researches. Exclude all those research studies which are not enough beneficial under our already defined categories.

**Results-oriented:** There should be result oriented researches included in this protocol. Target output of research must hold a good, effective and a solid approach and it must include an experimental method. Weak methodology / validated method researches must be excluded.

**Repetition:** Exclude the researches which are repeating in any context and only include those which are identical in context of research.

#### *2.1.2 Search Process*

Inclusion/ exclusion criteria, presented in the Section **2.1.1**, addressed that three databases (i.e. ACM, IEEE and Springer) have been selected to conduct this systematic literature review. Selected databases must include the high impact journals and conference proceedings. To get relevant results and to achieve search process, we have to use several search terms which is according to our defined criteria i.e. NLP, Test case generation using NLP etc. Results are summarized in Table **1** according to search terms relevant to different databases. We used different filter "2005-2014" for the search terms, so that to get the published researches from the year 2005-2014. We used some advanced search to control the search by searching only Journals in some cases. **Figure 3** shows the steps used in the search process.

- We select three data bases to carry out this research i.e. ACM, IEEE and SPRINGER.
- We write different search terms in specified databases and get 3195 search results.
- We basically eliminate 1955 research studies on the basis of their title as per our criteria of inclusion and exclusion.
- Then, we eliminate 519 research studies on the basis of their Abstract as per our criteria of inclusion and exclusion.
- Then, we took 325 research studies for the general study and by reading the relevant parts of research papers, we eliminate 158 research studies on the basis of our criteria of inclusion and exclusion. Remaining 167 research works are then selected for detailed study for our research.
- We did detailed study of 167 research papers and exclude 151 research works.

- We include 16 research works which are fulfilling our criteria of inclusion and exclusion.

**Table 1. Search terms and results**

| Sr. # | Term for Search | Results | | |
|---|---|---|---|---|
| | | SPRINGER | IEEE | ACM |
| 1 | NLP | 257 | 1742 | 1388 |
| 2 | Natural Language Processing | 1223 | 16023 | 6524 |
| 3 | Software testing using NLP | 416 | 22 | 701 |
| 4 | Software testing using Natural Language Processing | 2983 | 365 | 12125 |
| 5 | Automation of software testing using NLP | 40 | 4 | 14464 |
| 6 | Test case generation using NLP | 283 | 6 | 11736 |
| 7 | Test case generation using Natural Language Processing | 1755 | 36 | 12769 |
| 8 | Test case automation using NLP | 59 | 4 | 10537 |

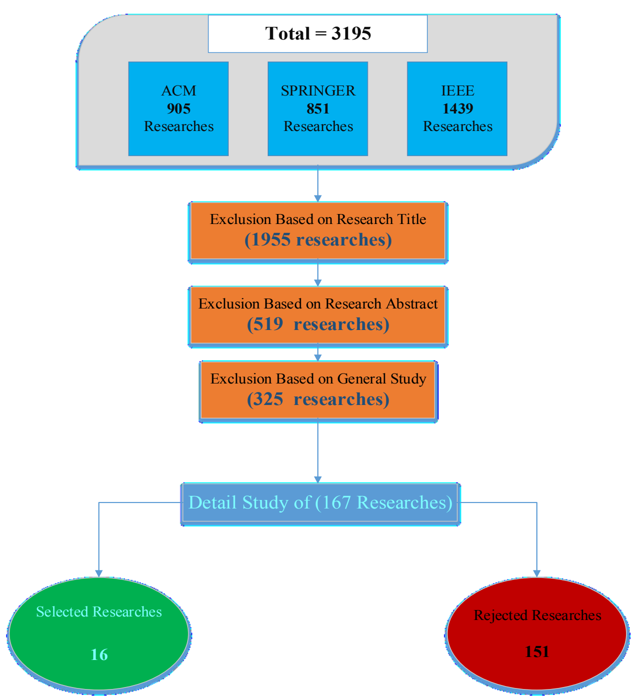


**Figure 3. Search process**

### 2.1.3 *Quality Assessment*

We established the criteria of quality to find out the significant results of the research works. The criteria also define each included research study and its findings:

- Data evaluation of the research work is dependent upon the solid facts without any forge statement.
- Selected research studies are validated through strong validation techniques i.e. case study etc.
- For the point of view of originality we included only those research works which are published in famous scientific databases i.e. SPRINGER, IEEE and ACM.
- As our objective was to include the most recent research studies. So, we used the filter of 2005 to 2016 to get researches which provide latest results. The yearly distribution of selected papers has been shown in **Figure 4**.


**Figure 4. Year-wise distribution of selected researches**

### 2.1.4 *Data Extraction and Synthesis*

We identify certain data extraction elements in order to get the answers of our research questions as given in Table **2**.

**Table 2. Data extraction / synthesis of selected researches**

| Sr.# | Description | Details |
|---|---|---|
| | **Data Extraction** | |
| **1** | Overview | The basic objective and proposal of included research |
| **2** | Data Collection | Qualitative or quantitative |
| **3** | Assumptions | Assumptions (if any) for validation of results |
| **4** | Results | Results acquired from the selected research |
| **5** | Validation | Method of validation used to validate its objective or proposal |
| | **Data Synthesis** | |
| **6** | NLP Techniques | Basic NLP techniques used to get output ( **Table** 4) |
| **7** | Tools | Tools used and developed in the NLP process (Table **5**) |
| **8** | Testing mechanism | Way to generate test cases (Table **3**) |
| **9** | Description/Overview of Research | Brief overview of research including introduction of proposed methodology etc. (Section **3.1**) |
| | **Bibliographic information** | |
| **10** | Bibliographic info. | Author, year of publication etc. |

# 3. RESULTS

## 3.1 Overview of Selected Researches

Harry M. Sneed et al [1] argued in is this paper that testing against NL requirements is one of the basic approaches for the acceptance & system testing. An unfamiliar organization with the application area performs the test independently. Written requirements are the thing which is applicable by the tester to view it before testing. It is necessary to analyzing those requirements and extracting test cases from the written requirements. So this paper consists of the presentation of requirements based testing and it is constructed on an industrial application. In this paper author developed the tool TEXT ANALYZER and used parsing as natural language processing technique. Author validated his approach through case study of mobile billing system and coca cola bottling.

Ravi Prakash Verma et al [2] presented an approach in which he has an argument that software testing play a vital role in the software system verification and makes sure the quality of the software system which is under development. Generation of software test cases is one of the most important challenging activities in this software testing. Test cases can be generated by several ways like sequence diagram, activity diagram and use case diagram which has also some limitations e.g. inability to generate test cases for nonfunctional requirements. There is a restriction of these steps in the acceptance testing and these step are not effective for verification of software systems. It will be difficult for the tester to test the system if the stated software requirements are in semi-formal or in formal methods. It requires an expert person to interpret the requirements. In this paper you find a propose way of approach for generating test cases from the software requirements in natural language by using NLP. In this paper author used the tool style parser and used parsing and POS tagging as natural language processing technique. Author validated his approach through experimental validation and test case generation algorithm.

Anurag Dwarakanath et al [3] argument was: Generation of test cases from the NL requirements is a challenging task when the requirement does not follow predefined structure. From a functional requirement document here in this paper, author give a details of tool for generating test cases. Author's methodology didn't impose any restriction on the structure of the sentence. A five steps process is followed for every requirement sentence and generate one or more test cases- 1) The testability of the sentence is analyzed by a syntactic parser whether it is testable or not; 2) A testable sentence with complexity is divided into several simple individual sentences; 3) Test intents are extracted from these simple sentence; 4) In order to generate positive test cases these test intents are grouped and sequenced temporally. An affirmative action of the systems is verified by a positive test case; 5) Boundary Value Analysis and other related techniques are used to generate negative test cases where these are applicable. Negative test cases basically identifies the system behavior in exception conditions. Author proposes a tool LITMUS in which he generated automated test cases. Here in this paper author presents experimental results of the tool and discusses the pros and cons of the tool. In this paper author used the tool link grammar and used parsing as natural language processing technique. Author validated his approach through industrial requirement dataset.

Chunhui Wang et al [4] claimed in this paper that test cases are generated from the natural language functional requirements which are error prone. Although there are a lot of researches in which test cases are generated by natural language plain text automatically but still human intervention is required there. Author proposed UMTG (use case modelling for system tests generation) in this paper, which automatically generate the test cases from the user requirements. In this paper author used the tool GATE and developed the tool UMTG and used parsing, POS tagging and tokenization as natural language processing technique. Author validated his approach through body sense case study.

Edgar Sarmiento et al [5] made an argument in this paper that software testing is usually time consuming task in the complex projects. Author argument was requirement engineering artifacts are start points for the software product development and author stated that mostly requirements are written in natural language so he presented a tool that implements the test case generation from NL requirements in this paper. Authors proposed tool translates the natural language requirements into the models so that to implements the testing. This approach is easy to use and is not time consuming approach. Author stated that his approach is cost effective too. In this paper author developed the C&L tool. Author validated his approach through test case generator algorithm and also validated his approach by creating test cases and activity diagram.

Monalisha Khandai et al [6] argument in this paper is: Concurrency testing is a bit difficult task due to arbitrary interference. And when testing the concurrent systems explosion of test case become a large issue. There are basically two characteristics of concurrent systems. One is synchronization and the other one is deadlock. Due to both characteristics systematic testing of the concurrent system become possible. Author presented an approach of generating test cases from the concurrent systems by the help of sequence diagram in this paper. Author presented approach converts the sequence diagram into the concurrent graphs and then these are traversed by some techniques e.g. depth first search, breadth first search by using a sequence path criteria. In this way author controls the explosion problem of the test cases. Author validate his approach through the ATM case study and he used message sequence path algorithm.

Roaa Elghondakly et al [7] presented a testing approach which generate test cases from the requirements is known as Requirements-based testing. In software developments life cycle these requirements represents the initial phase. All software's are developing on the basis of some basic requirements. Any loophole in natural language requirements leads in a leading mistake in the coming phases. There are so many software development models such as waterfall model, agile model, etc. here in this paper we suggest a novel automated progress to develop test cases from requirements. Sources of the requirements may be the different models either from waterfall model (functional or nonfunctional) or agile model. The proposed approaches to develop test cases as different type of requirements are covered such as SRS documents, non-functional requirements, and user stories. For data generation and validation this proposed approach use text mining and execution of symbolic methodology, where a multi-disciplinary domains is developed based on knowledge.

In this paper, Christopher B. Harris et al [8] argument was: for verification purpose, correctness properties are designed by engineer who has a duty to address the analysis of documentation in application specific integrated circuits (ASIC). While 60% of constructing endeavors is spent on authentication and testing actions for this process. A natural language based translation system represented for generating formal attribute grammar which automatically develops sentence correction verifications in

Computation Tree Logic (CTL) of Hardware Description Language (HDL). This ideology is standardize adopting cross check of information take away of an authentication the PCI bus stipulation enclosed with Texas-97 authentication criterion suite in addition to alter English successfully to legitimate CTL in 91% of the referral. The autogenously CTL realties are comparing with the CTL realties adjusted to criterion along with determining of the equivalency of developed and benchmark properties by using model checking. Some of the properties are fond additive footings that results in CTL intimately reflect fabricators captive though majority of properties are same. In this paper, Author used parsing and tokenization as NLP technique.

Matthias Schnelte et al [9] stated that dynamic testing is considered the most used QA technique in machine-driven industry. So the automation of test case generation becomes so necessary. In our project we focused more on this automatic generation of test from the requirement specifications. We try to insert the approach closely as much as possible into previous workflows with natural language such as specifications and requirements which are written in natural language. For assisting this we select a controlled natural language for our automotive domain. A formal model is developed by translating after requirements acquisition. This model has an effective reach ability analysis and it allows a rich temporal behavior description. After that we use a partial order planning for creating positive and negative tests. The results of the test case are capable to control the timing behavior. Moreover, the test cases can be show in a soft way by which readers can formalize them. Author used POCL algorithm and implements his approach by implementing a data set.

In this paper Mehdi Malekzadeh et al [10] showed the enlargement of Automatic Test Case Generator (ATCG) for proofing the system on inviolability- critical spyware by applying brain wave of stipulation foundation trialing. Stipulation of systems unfinished trial in a regular stipulation language articulation together with reasons and trappings are cardinally extracted by the ATCG along with cause-effect tabulation stipulation model. With the combined assistance of Boolean operator technique test cases bring-on by exploitation cause-effects graphical record software testing methods in details. When redundant trial studies are fabricated in ATCGs, always fruitions are not to be found in perfect. Our test at ATCGs, Practical fruition exposed that it produces quality survey along with ignoring the redundant trial studies. Author used tokenization as natural language processing technique. Author validate his approach through boiler control monitoring system and used BOR algorithm.

In this paper, Dante Torres et al [11] presented his contribution of generating test case descriptions from the requirements specifications. Author presented a tool named SpecNL which generates a description of software test cases in natural language from test case specification. The prime objective of this tool is assisting the engineers to operate the manual test. This tool is a part of mobile phone software projects for automate testing. SpecNL is an intercrossed system because it is a combination of linguistic information and output guide in the generation task. Satisfactory results are revealed by implemented prototype experiments. In the software engineering area this is creative and progressive application of NLP. Author validate his approach by the analysis of Motorola test engineers. Author also used parsing as natural language processing technique.

Matthias Riebisch et al [12] stated that testing complicated and large number of requirements for the computer based systems is very rigorous and difficult task. Therefore requirements are important for generating test by the use of models. The efforts and risk of the error can be considerably reduced by providing support at time of generation of the test cases. At the time of modeling the understanding of model improvement was better in past, input requirements specification analysis mostly includes of natural language texts- still indicate a constriction. Explanation of closing gaps of manual structuring tests techniques and automated lingual based techniques are given in this paper. A glossary and analysis of descriptions of integration in use cases are source of the suggestions. Elements of other models and traceability links via feature models are analyzed. Increasing of the level of tool support, higher efficiency results of the test cases are advantages of both techniques. Author verified his approach by applying templates of SOPHIST and developed the XSLT based prototype.

Harry M. Sneed et al [13] proposed an organized method in this paper to test the web service behavior in NL by using the keywords. Automatically analysis aligning extracting test cases for testing the web service is the advantage of the document. Test study shows what to test, but also say how the test take places such as on the basis of what data values the test is done. Author's objective in this paper is joining specification in a document used for specifying the testing task. Comparison of functional requirements table is generated. On the other hand, by addressing the mentioned functions a test case is initiated. Requested tests are produced from the test script that are generated from test case table. The test case script are used for validation of the system. From the original service requirements specification all the needed information are taken for request generation and response validation. Preparation of requirement document is shown is the case study- order entry service and author used parsing as natural language processing technique.

In this paper, MARIANO CECCATO et al [14] suggested more than one technique and tool for the automatic generation of test cases. In general, proposed tools are evaluated on the basis of fault informative or extend, capability, but consideration has rejected for the impacts on manual fault identification activity. There is a question that whether automatically developed test cases are as effective as manual written tests in supporting identification of debugging. A three experiments family (five – replication that repetition of experiments condition for estimating the variability of the of the phenomenon) with human (55 in total number) for assessing that is the characteristics are related to AGTC, which make them lower visible and clarity (for example unclear test scenarios, insignificant identifiers) have an effect on the effectiveness and efficiency of debugging. The first two tests make a comparison of different test cases developing tools (Rabdoop vs. Evosuite). The role of code identifiers in test cases are investigated by the third experiment (fake vs. original identifiers), since the latter enclose insignificant (a changes condition) identifies the experiments which is a leading differences between manual and automatically developed test cases. Usefulness for debugging is same in both automatic and manual test case. It can be mention that, in our finding, we observe that automatically tests cases are more useful to the lower experienced developer because of their less inactive and dynamic complication. Author did experimental verification of the test cases and used parsing as natural language processing technique.

Gustavo Carvalho et al [15] is this paper arguments that use of formal models as inputs are addressed in automated test generation strategies. For example detecting and correcting errors during the requirement appearances. Software Cost Reduction is constructed and for giving permission to test case generation. However, SCR syntax is insignificant for both who is known and unknown user of it. It is our suggestions for a strategy in this paper for developing test cases from natural language requirements using SCR as a medium. To bypassing textual expression of records necessities are written on the basis of a Controlled Natural Language, Every syntactic valid requirement represents into a semantic presentation from which an SCR specification is derived. Then we apply the T-VEC tool for generating test from SCR. On the basis of requirements and manual record test variables on evaluation of our strategy which are supplied from aviation industry by our partners. Our game-plan generates originally almost 85% of the vectors within exactness of 100%. We found a modification of 84% within only 2s generating time periods. Author validated his approach through analysis and used parsing as natural language processing technique.

Avik Sinha et al [16] argued that the modularity and customer based centralization approach for use cases render them to the reliable system requirement stimulation, specifically for repetitive software improvement process as in active scheduling. a variety of regulations exists for the use cases process and subject, but recently it require specialized training and hard managed requirement elicitation process for enforcing formality to such regulations in the industry. Anyway, for wrong development schedules, respective entity keeps away themselves from such type of extended processes and stop interpreting use cases in an unplanned fashion within a poor guidance. The results of use cases with poor quality standard are only fit for downstream software activities. An approach is developed by authors for automated and "edit time" review of use case of the development and analyzing of models on the basis of use cases. Author's proposed models are constructed with linguistic and functional properties of both use case and system under discussion. Author presented a suitable model analysis technique that provide such models for validating use cases at the same time for their style and content. Such model analytical techniques may be the compound with robust NLP techniques for developing integrated improvement environments for use cases authorization, which we do in Text2Text. During use in industrial setting, it results in better agreeability of use cases, improving production, and afterward in higher quality of test cases. Author validated his approach by using 5 case studies and also used parsing, POS tagging and tokenization as natural language processing technique.

To this point, we summarize the relevant details of selected research works. The precise details, required to get the answers of the research questions, have been presented in subsequent sections.

## 3.2 Test Case Generation Scenarios

Generally, test cases can be generated through NLP techniques in six categories as defined below:

**Test Values to Test Cases:** It is used to simulate symbolic values of variables, instead of actual values, to act as input to the system under test, in order to find the expected output according to those inputs. For example describes the values which will be used for test case generation like Integer, Boolean Expressions etc.

**Test Intents to Test Cases:** Test Intents map to the aspects on which the requirement is to be tested. Test Intent as the smallest segment of a requirement sentence that conveys enough information about the purpose of the test. A Test Intent has a Subject, an Action and (optionally) an Object.

**Test Input and Scenarios to Test Cases:** Scenario is a language used to help the understanding of the requirements of the application. It is easy to understand by the developers and clients. Scenarios represent a partial description of the application behavior that occurs in a specific geographical context.

**Test Condition to Test Cases:** Test Condition is the behavior of system which should be first checked before generating test cases.

**Multiple Ways to Test Cases:** Studies which generate test cases using more than one way lie under this category. E.g. [5] uses Test Value to Test Cases, Test Intents to Test Cases, Test Inputs to Test Cases and Test Condition to Test Cases.

**Unevaluated:** No test cases were generated but method was proposed to make a mechanism to generate test case. E.g. in [8] methodology is given to test the verification of developed system but no output is generated. The test case generation scenarios of selected research works are summarized in Table **3**.

## 3.3 Natural Language Processing Techniques

NLP has the following techniques: tokenization, POS tagging, Chunking and Parsing. These techniques are often used to process a text. We collected results of our researches which are based on these techniques. Some of the researchers utilized all the techniques, some use one or two and three, authors in [5, 6 and 9] used nothing of them. The researches which didn't use any technique have used some tool to perform natural language processing which by default includes these techniques. For example, [5] developed C&L tool which has the capabilities of parsing the sentence. The utilization of major NLP steps, in the context of text cases generation, has been summarized in

**Table** 4.

**Table 3**. **Summary of test case generation scenarios**

| **Test Case Generation Scenarios** | **Test Value to Test Cases** | **Test Intents to Test Cases** | **Test Input and Scenarios to Test Cases** | **Test Condition to Test Cases** | **Multiple ways to Test cases** | **Unevaluated** |
|---|---|---|---|---|---|---|
| **References** | [2], [7], [15],[16] | [3] | [1], [4], [6], [10], [12], [13] | [9] | [5] | [8], [11], [14] |
| | 4 | 1 | 6 | 1 | 1 | 3 |

| Total | 16 |
|---|---|

**Table 4. Identification of NLP techniques**

| Sr. # | NLP Steps | Relevant Researches | Total |
|---|---|---|---|
| 1 | Tokenizing alone | Mehdi Malekzadeh et al[10], | 1 |
| 2 | POS Tagging alone | Matthias Riebisch et al[12] | 1 |
| 3 | Parsing alone | Harry M. Sneed et al [1], Anurag Dwarakanath et al [3], Roaa Elghondakly et al[7], Dante Torres et al[11], Harry M. Sneed et al[13], MARIANO CECCATO et al[14], Gustavo Carvalho et al[15], Avik Sinha et al[16] | 8 |
| 4 | POS Tagging + Parsing | Ravi Prakash Verma et al [2] | 1 |
| 5 | Tokenizing + Parsing | Christopher B. Harris et al[8] | 1 |
| 6 | Tokenizing + POS tagging + Parsing | Chunhui Wang et al [4] | 1 |

## 3.4 Tools and Algorithms Identification

The identified tools and algorithms are presented in Table **5**.

**Table 5. Identified tools and techniques**

| Sr.# | Paper | Tool Proposed | Tools Utilized | Validation Method | Algorithm Used |
|---|---|---|---|---|---|
| **1** | Harry M. Sneed et al [1] | Text analyzer | N/A | Case study (Coca Cola bottling, Mobile billing system) | N/A |
| **2** | Ravi Prakash Verma et al [2] | N/A | Style Parser [19] | Experimental validation | Test case generation algorithm |
| **3** | Anurag Dwarakanath et al [3] | Litmus | Link Grammar parser [20] | Validation by Industrial requirements dataset | Algorithm to merge and generate test cases |
| **4** | Chunhui Wang et al [4] | UMTG | GATE [17] | Case Study (Body Sense) | Building path condition algorithm |
| **5** | Edgar Sarmiento et al [5] | C&L | N/A | Validation by creating activity diagram and test cases | Test case generator algorithm |
| **6** | Monalisha Khandai et al[6] | N/A | N/A | Case study (ATM) | Message sequence path algorithm |
| **7** | Roaa Elghondakly et al[7] | N/A | N/A | web publishing system SRS, Library management system, Data Management system | N/A |
| **8** | Christopher B. Harris et al[8] | N/A | NLTK [18] | Texas-97 Verification Benchmark suite | N/A |
| **9** | Matthias Schnelte et al[9] | N/A | N/A | Requirements DataSet | POCL algorithm |
| **10** | Mehdi Malekzadeh et al[10] | ATCG | N/A | Boiler control and monitoring system | BOR algorithm |
| **11** | Dante Torres et al[11] | SpecNL | ADL translator [21] | Analysis by Motorola's test engineers | N/A |
| **12** | Matthias Riebisch et al[12] | XSLT-based prototype tool | N/A | Validation by applying templates of the SOPHIST | N/A |
| **13** | Harry M. Sneed et al[13] | Web service testing tool | TestSpec [22] | Case study- order entry service | N/A |

| | | | | | |
|---|---|---|---|---|---|
| **14** | MARIANO CECCATO et al[14] | N/A | Randoop [24], EvoSuite [23] | Experimental Validation | N/A |
| **15** | Gustavo Carvalho et al[15] | SCR-Generator | T-VEC tool [25], CNL Parser [26] | Analysis based | SCR algorithm |
| **16** | Avik Sinha et al[16] | Text2Test IDE | N/A | Experimental Validation | Model Analysis algorithm |

# 4. ANSWERS OF RESEARCH QUESTIONS

**RQ1**: What research works have been reported from 2005 to 2016 where NLP techniques have been utilized to generate automated test cases from initial requirements?

**Answer:** As per selection and rejection criteria, we identified 16 researches from year 2005-16 (Section**2.1.1**) where NLP techniques have been utilized for automated test case generation from plain text. The selected research works are summarized in Section **3.1**.

**RQ2**: How major NLP steps like Tokenization, POS tagging, Chunking and Parsing have been applied for test cases generation?

**Answer:** The utilization of major NLP steps, in the context of text cases generation, has been summarized in

**Table** 4 where NLP techniques such as tokenization, POS tagging, chunking and parsing have been applied for the test case generation.

**RQ3**: What are the leading NLP tools and algorithms, utilized or proposed by the researchers, in order to generate test cases from initial requirements?

**Answer**: On the basis of systematic literature review, 18 tools and 9 algorithms are identified for the test case generation from the natural language text given in Table **5**.

# 5. DISCUSSION

There are 16 researches for the discussion. Selected parameters are relevant to researches for analysis. Simplicity, complexity, output nature and tool support are the parameters selected. E.g. [1, 2, 3, 4, 6, 7, 8, 10, 12, 15] outputs test cases. In research [5] model-based test cases are generated. In [8] author generated Computational tree logics. In [13] and [11] tests for web service and test case descriptions are generated respectively while in research [16] nothing is generated but author did test cases' inspection and gave an analysis.

In our SLR, we have analyzed different studies on basis of simplicity and complexity. Like in research [4] tokenization, parsing and POS tagging is used as NLP technique, our analysis says it is a complex approach to process any Natural Language requirements. As In [2] author applies just parsing and POS tagging to process Natural Language text as NLP technique which is less complex approach with respect to [4]. In [8] POS tagging and tokenization is used as NLP technique which is also a less complex approach.

Our analysis on complexity and simplicity parameters derives that in [1, 3, 7, 11, 13, 14, 15 and 16] only parsing is applied which is much simpler approach with comparison to others. On analysis of another parameter of tool support we analyzed the test case depth and validation process. In [1] author developed Text Analyzer and validates his approach with mobile billing system and coca cola bottling case studies. In study [2] style parser is used and author applied test case generation algorithm for test case generation and experimental validation is performed. In [3] author developed

Litmus tool using link grammar parser for test case generation and through data set of industrial requirement validate his technique. UMTG is developed in [4] by using Gate tool and validated by a Body sense case study to generate test cases. In study [10] ATCG tool is developed by using an algorithm BOR and validated by boiler control and monitoring system case study to generate test cases. In [11] SpecNL is developed by using ADL translator and validated by a Motorola test engineers for the generation of test cases. While In research [12] XSLT based prototype tool is proposed by author and generation of test cases is validated by applying templates of SOPHIST. TestSpec tool is used to develop Web service testing tool in [13] and validated by order entry service case study. In [14] EvoSuite and Randoop are used by author and comparison is performed through experimental validation. In [15] CNL parser tool and T-VEC are used to develop SCR generator by applying SCR algorithm which is used to generate test cases which are validated by an analysis. While in [16] author performed experimental validation on developed tool Text2Test IDE using model analysis algorithm, he also performed inspection of test cases.

Our analysis shows the test depth of all researches, [2], [5], [7], [15], [16] derive test values into test cases. [5] And [9] derive test conditions into test cases. [5], [6], [10], [16], [34] took test input and scenarios to convert into test cases. While [3] and [5] generated test cases form test intents.

# 6. Benefits Aims & Limitations

As we know that NLP shows some promising results to identify the desired information from bulky plain text so benefits of this research study are: We investigated the application of natural language processing technique to generate the test cases from natural language plain text and able to investigate 18 tools, 4 techniques and 9 algorithms for test case generation.

Aim of this research study is to investigate the latest researches where NLP techniques have been put into practice to generate automated test cases from raw documents. This is the first real investigation in this domain to the best of our knowledge.

One of the major limitation of this study is that we just extracted all the researches from three databases ACM, IEEE and Springer which are the most authentic research repositories. However, we believe that it would be more valuable to include further reputed scientific databases.

# 7. CONCLUSION AND FUTURE WORK

This paper investigates Nature Language Processing (NLP) techniques to generate automated test cases from initial requirements document. A Systematic Literature Review (SLR) has been carried out to perform this study. 16 research works, published during 2005-2014, have been selected on the basis of review protocol. Consequently, 6 NLP techniques have been identified to generate automated test cases. Moreover, 18 tools have been presented where 10 tools are proposed by the researchers and 8 existing tools are utilized to perform certain NLP steps. Furthermore, 9 NLP algorithms have been identified in the context of test case generation. The NLP techniques, tools and algorithms, presented in this paper, are highly valuable for the researchers and practitioners of the domain.

Although this study presents comprehensive overview of NLP techniques and tools to generate automated test cases from preliminary requirements, it is still required to perform comparative study of identified tools and algorithms in order to highlight their strengths and weaknesses. Therefore, in next article, we intend to perform comparative analysis of identified tools with particular parameters like usability, support for major NLP steps (i.e. Tokenization, POS tagging, Chunking and Parsing) and license type (Open source or proprietary) etc.

## 8. REFERENCES


[1] Harry M. Sneed, “Testing against natural language Requirements”, Seventh International Conference on Quality Software, 2007 IEEE

[2] Ravi Prakash Verma, Dr. (Prof.) Md. Rizwan Beg, “Generation of Test Cases from Software Requirements Using Natural Language Processing”, 6th International Conference on Emerging Trends in Engineering and Technology, 2013 IEEE

[3] Anurag Dwarakanath and Shubhashis Sengupta, “Litmus: Generation of Test Cases from Functional Requirements in Natural Language”, Springer-Verlag 2012, pp. 58-69

[4] hunhui Wang, Fabrizio Pastore, Arda Goknil, Lionel Briand, Zohaib Iqbal, “Automatic Generation of System Test Cases from Use Case Specifications”, ISSTA, 2015/ ACM

[5] Edgar Sarmiento, Julio Cesar Sampaio do Prado Leite, Eduardo Almentero, “C&L: Generating Model Based Test Cases from Natural Language Requirements Descriptions”, 2014 IEEE

[6] Monalisha Khandai, Arup Abhinna Acharya, Durga Prasad Mohapatra, “A Novel Approach of Test Case Generation for Concurrent Systems Using UML Sequence Diagram”, 2011 IEEE

[7] Roaa Elghondakly, Sherin Moussa, Nagwa Badr, “Waterfall and Agile Requirements-based Model for Automated Test Cases Generation”, Seventh International Conference on Intelligent Computing and Information Systems (ICICIS'15), 2015 IEEE

[8] Christopher B. Harris, Ian G. Harris, “Generating Formal Hardware Verification Properties from Natural Language Documentation”, 9th International Conference on Semantic Computing (IEEE ICSC 2015), 2015 IEEE

[9] Matthais Schnelte, “Generating Test Cases for Timed Systems from Controlled Natural Language Specifications”, Third International Conference on Secure Software Integration and Reliability Improvement, 2009 IEEE

[10] Mehdi Malekzadeh, Raja Noor Ainon, “An Automatic Test Case Generator for Testing Safety-Critical Software Systems”, 2010 IEEE

[11] Dante Torres, Daniel Leitão, Flávia Barros, “Motorola SpecNL: a Hybrid System to Generate NL Descriptions from Test Case Specifications”, Proceedings of (HIS'06), 2006.

[12] Matthias Riebisch, Michael Hübner, “Traceability-Driven Model Refinement for Test Case Generation”, 12th International Conference and Workshops on the Engineering of Computer-Based Systems (ECBS’05), 2005 IEEE

[13] Harry M. Sneed, Chris Verhoef, “Natural Language Requirement Specification for Web Service Testing”, 2013.

[14] MARIANO CECCATO, ALESSANDRO MARCHETTO, LEONARDO MARIANI, CU D. NGUYEN, PAOLO TONELLA, “Do Automatically Generated Test Cases Make Debugging Easier? An Experimental Assessment of Debugging Effectiveness and Efficiency”, ACM Transactions on Software Engineering and Methodology, Vol. 25, No. 1, Article 5, Pub. Date: November 2015

[15] Gustavo Carvalho, Diogo Falcão, Flávia Barros, Augusto Sampaio, Alexandre Mota, Leonardo Motta, Mark Blackburn, “Test Case Generation from Natural Language Requirements based on SCR Specifications”, SAC’13 March 18-22, 2013, Coimbra, Portugal, ACM 2013

[16] Avik Sinha, Stanley M. Sutton Jr., Amit Paradkar, “Text2Test: Automated Inspection of Natural Language Use Cases”, Third International Conference on Software Testing, Verification and Validation, 2010 IEEE

[17] H. Cunningham, D. Maynard, K. Bontcheva, V. Tablan, C. Ursu, M.Dimitrov, M. Dowman, N. Aswani, and I. Roberts, Developing Language Processing Components with GATE, University of Sheffield, 2005.

[18] Bird, S., Klein, E., Loper, E., 2009. Natural Language Processing with Python. O’ReillyMedia

[19] Ratnaparkhi, A. (1998). Maximum Entropy Models for Natural Language Ambiguity Resolution. Ph.D. thesis, University of Pennsylvania.

[20] Sleator, D.D.K., Temperley, D.: Parsing English with a Link Grammar. In: Third parsing Workshop (1993)

[21] ADL project. Assertion Definition Language project website. 2000. Available in: http://adl.opengroup.org/index.html. Accessed in March 06, 2006.

[22] Sneed, H.: “Testing against natural language Requirements”, 7th IEEE International Conference on Software Quality (QSIC2007), Portland, Oct. 2007, p. 380

[23] G. Fraser and A. Arcuri. 2011. Evolutionary generation of whole test suites. In Proceedings of the 11th International Conference on Quality Software (QSIC’11). 31–40.

[24] C. Pacheco and M. D. Ernst. 2007. Randoop: Feedback-directed random testing for Java. In Companion to the 22nd ACMSIGPLAN Conference on Object-Oriented Programming Systems and Applications Companion (OOPSLA’07). ACM Press, New York, 815–816.

[25] M. Blackburn, R. Busser, and J. Fontaine. Automatic Generation of Test Vectors for SCR-style Specications. In Proceedings of ACSAC, 1997.

[26] J. Allen. Natural Language Understanding. Benjamin/Cummings, 1995

[27] Kitchenham,B.,2004.ProceduresforPerformingSystematicReviews.KeeleUniversityTR/SE-0401/NICTA,TechnicalReport0400011T.

[28] Marie Meteer, Bensiin Borukhov, Michael Crivaro, Michael Shafir, Attapol Thamrongrattanarit “MedLingMap: A growing resource mapping the Bio-Medical NLP field”, Proceedings of the 2012 Workshop on Biomedical Natural

Language Processing (BioNLP 2012), pages 140–145, Montreal, Canada, June 8, 2012

[29] Ashfa Umber, Imran Sarwar Bajwa, M. Asif Naeem, "NL-Based Automated Software Requirements Elicitation and Specification", ACC 2011Proceedings, Part II, pp 30-39

[30] Deva Kumar Deeptimahanti, Muhammad Ali Babar, "An Automated Tool for Generating UML Models from Natural Language Requirements", International Conference on Automated Software Engineering, IEEE/ACM, pp. 680-682

[31] ACM, Last Accessed August 2016.http://dl.acm.org/

[32] IEEE Scientific database, Last Accessed, August 2016.http://ieeexplore.ieee.org

[33] Springer, Last Accessed August 2016.http://link.springer.com